FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES FAST School of Computing · Karachi Campus · Department of Software Engineering
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	**Aiman Farooqui**	**24k-3077**	**SE-5A**	Sole developer: all modules, experiments and documentation

Project Title	**Test Suite Optimizer: Running fewer test without missing bugs**
Project Idea	Own Idea (needs approval)
GitHub Repository URL	**https://github.com/Aiman-005/Test_suite_Optimizer**
Date of Submission	**15th week**

1. Project Title and Summary

Test Suite Optimizer Reduces Missing Bugs and Runs With Less Tests. Software projects maintain extensive sets of manual tests (a test suite) that are re-executed after every change of code to ensure defects are fixed, and on actual projects, this can take a considerable amount of time to complete. This project creates a tool that helps decide which tests to run when you have a limited amount of time, like running only 30% of the usual test time.The tool aims to make sure as much of the code as possible is still tested. The target audience for this tool is software developers and teams that require prompt feedback on every change. A simple program can only run all the tests or stick to a set rule.But an AI looks at choosing which tests to run by exploring a huge number of possible groups.It uses a local search method to quickly find a group that works really well in just a few seconds. The selection is screened for by intentionally introducing small bugs into the code and assessing the number of bugs that the smaller test set still catches.

2. Problem Statement and Motivation

Regression testing means running a project's tests again after every change. Normal automated testing runs all tests every time, so on big projects each update can take minutes or hours, and many tests check the same lines of code repeatedly. Choosing a smaller set of tests is a combinatorial problem: a project with 300 tests has about 2^300 possible subsets, more than the number of atoms in the universe, so checking every option is impossible. Formally, it is a budgeted maximum coverage problem, which is NP-hard, and because tests overlap, the value of adding a test depends on what has already been selected. Teams face this daily in continuous integration and cope by running all tests, running slow ones overnight, or using a "smoke test" set that soon becomes outdated. Solving it gives developers faster feedback without letting bugs slip through.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Percentage of code lines covered by the selected tests, total running time kept within the budget, and percentage of seeded bugs (mutants) caught; secondary: optimizer run time.
Environment	One open-source Python project's test suite, represented by a coverage matrix (which test covers which lines), each test's estimated duration and a fixed time budget.
Actuators	Adding a test to the selection, removing a test from it, or swapping one selected test for an unselected one; finally, outputting the selected set of tests to run.
Sensors	The coverage matrix, plus repeated noisy timing measurements of each test that are filtered into duration estimates; from these the agent computes the coverage and total time of any candidate selection.

3.2 Environment and Agent Type

Property	Classification
Observability	Fully observable
Determinism	Deterministic
Change over time	Static
State and actions	Discrete
Agents	Single Agent
Agent type	Utility based Agent

3.3 Problem Representation

Element	Your representation
State representation	Local search: a binary vector x of length n; x_i = 1 means test i is selected. Exact search (UCS, A*): the set of tests chosen so far. CSP: one 0/1 variable per test.
Actions / moves	Local search: flip one bit (add or remove a test) or swap one selected test for an unselected one. Exact search: add one test that covers the lowest-numbered line not yet covered. CSP: assign 0 or 1 to the next variable.

Initial state	Hill climbing and simulated annealing: a random feasible selection generated with a fixed seed. Greedy, UCS, A* and CSP: the empty selection.
Goal or terminal test	Local search has no goal state: hill climbing stops when no neighbour improves fitness, simulated annealing when the temperature falls below a minimum or an iteration limit is reached. Exact search: every coverable line is covered. CSP: every line has a selected covering test and total time ≤ budget.
Cost, utility, fitness or evaluation	Fitness (local search) = fraction of coverable lines covered if total time ≤ budget, otherwise coverage minus a penalty proportional to the time over the budget; ties favour lower time. Path cost (UCS, A*) = total seconds of the selected tests.

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Hill climbing with random restarts, simulated annealing (Lab 06)	Core optimizers for the full suites: improve a selection step by step under the time budget.	Own code
2	Uniform-cost search (Lab 04) and A* (Lab 05)	Exact search on small sub-suites: find the cheapest selection that covers every line, giving the true optimum to compare local search against.	Own code
3	Constraint satisfaction (Lab 07)	Decides whether full coverage fits a given budget; plain backtracking compared with backtracking + MRV + forward checking.	Own code
4	Kalman filter (Lab 09)	Turns repeated noisy timings of each test into a reliable duration estimate used by every optimizer.	Own code
5	Exploratory data analysis (Lab 10)	Test-duration distribution, coverage overlap between tests, and lines covered by only one test.	Own code using Pandas

#	Technique	Role in the project	Own code or library
B	Baseline: cost-aware greedy and random selection	Greedy repeatedly adds the test with the most new lines per second until the budget is used; random adds tests in random order.	Own code

Full suites have up to 2^300 subsets and no single goal state, so local search improves one complete selection step by step. Its fitness is the percentage of lines covered, with a penalty for exceeding the budget; simulated annealing accepts worse moves with probability e^(−Δ/T) under geometric cooling, so it can escape local optima where greedy and hill climbing get stuck. On small sub-suites, UCS finds the cheapest full-coverage selection, and A* uses h = uncovered lines × the lowest seconds-per-new-line of any remaining test; it never overestimates, because every uncovered line must still be paid for at no less than that rate. The CSP checks whether full coverage fits a budget, with MRV and forward checking pruning dead ends early. A one-dimensional Kalman filter models each test's true duration as nearly constant and each run as a noisy measurement of it.

4.1 Planned Class Design

- **TestSuite**: tests, durations, coverage (test → set of line IDs); methods coverage_of(sel), time_of(sel).
- **SelectionProblem**: suite, budget; methods neighbours(state), fitness(state), is_feasible(state).
- **Optimizer** (abstract): abstract optimize(problem). **GreedyOptimizer**, **HillClimbingOptimizer** and **SimulatedAnnealingOptimizer** each override optimize().
- **SearchSolver** (abstract): frontier, explored set, nodes_expanded; abstract priority(node). **UniformCostSearch** returns g(n); **AStarSearch** overrides it to return g(n) + h(n).
- **CoverCSP**: solve(use_mrv, forward_checking), counts backtracks.
- **KalmanDurationEstimator**: update(measurement) returns the new duration estimate.

5. Scope

#	Feature	Why it is essential
1	Data collection for 3 open-source Python projects: per-test line coverage and 10 repeated timing runs saved as CSVs	Provides the real problem instances and the noisy measurements for the filter.
2	Kalman-filtered duration estimates, plus TestSuite and SelectionProblem with greedy and random baselines	Formulates the problem precisely on reliable durations and gives the comparison baselines.
3	Hill climbing and simulated annealing in my own code, with fitness logged every iteration	The core AI technique (Lab 06) and the source of the convergence charts.
4	Exact solvers on small sub-suites: UCS and A* with node counts, CSP with backtrack counts	Gives the true optimum to measure how close local search gets (Labs 04, 05, 07).

5	Fault-detection check: small bugs (mutants) inserted into each project and a kill matrix of which tests catch them	Proves the smaller test set still finds bugs, not only that it covers lines.

Out of scope

Because this is a solo project, the genetic algorithm is excluded; it would add a third local search method but no new technique family. Adversarial search (Lab 08) does not apply, since test selection has no opponent, and supervised learning is not used because there is no natural labelled prediction task in the problem. Exact search is run only on small sub-suites, because it cannot scale to full suites. Also excluded: change-aware selection based on the latest code edits; packaging as a pytest plugin, GitHub Action or any deployment; running tests live from the dashboard (it reads pre-collected data only); branch coverage; handling of flaky tests; and non-Python projects.

6. Data or Environment Source

Source name and URL	Public open-source Python projects on GitHub; candidates: TinyDB (https://github.com/msiemens/tinydb), marshmallow (https://github.com/marshmallow-code/marshmallow), Click (https://github.com/pallets/click), toolz (https://github.com/pytoolz/toolz), boltons (https://github.com/mahmoud/boltons). The final three are the first whose full test suites pass locally.
Type	Self-collected
Access requirements	No key needed
Approximate size	About 100–1,500 tests and 1,000–10,000 lines of code per project; 10 timing runs per suite; up to 150 mutants sampled per project; processed CSVs of a few MB in total.
Licence / terms of use	Permissive open-source licences (e.g. MIT, BSD-3-Clause); each licence is checked before use, and each project and its exact commit are cited in the README and report. Only derived data is stored, not the projects' source code.
Offline fallback	Collection is done once; the processed coverage, timing and kill-matrix CSVs are committed to `data/processed/`. Everything else reads only these local files and never uses the network, so the project runs from a clean clone without internet. The collection script records each project's commit hash so the data can be regenerated.

7. Evaluation Plan

What will be compared	Full suites: random, greedy, hill climbing, simulated annealing. Small sub-suites: UCS vs. A*, plain backtracking vs. MRV + forward checking, and every optimizer vs. the exact optimum. Raw timings vs. Kalman estimates.

Metric(s)	Coverage (%); mutants caught (%); time vs. budget; run time; nodes expanded; backtracks; gap to optimum (%); budget overruns when re-timed on a held-out run.
Baseline	Greedy and random selection; UCS for A*; plain backtracking for the CSP; raw timings for the Kalman filter.
Test instances / test set	3 projects × 7 time budgets (10–70% of full suite time) = 21 full-suite instances, plus 10 seeded sub-suites of 12–15 tests per project = 30 small instances.
Number of runs and seed	3 projects × 7 time budgets (10–70% of full suite time) = 21 full-suite instances, plus 10 seeded sub-suites of 12–15 tests per project = 30 small instances.
Planned charts (at least three)	(1) Convergence of hill climbing and simulated annealing; (2) coverage vs. budget; (3) fault detection vs. budget; (4) nodes expanded (UCS vs. A*) and backtracks; (5)gap to optimum; (6) Kalman estimate vs. raw timings.

8. Division of Work

Member	Modules / components owned	Approximate share
Aiman Farooqui 24k-3077	All modules, experiments, dashboard and documentation	**100%**

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date
1	Coverage CSVs and 10 timing runs for all 3 projects; EDA notebook on durations and overlap	28 Oct 2026
2	Kalman duration estimates with a filtered-vs-raw plot; random and greedy solve all 21 instances	3 Nov 2026
3	Hill climbing and simulated annealing solve all 21 instances with fixed seeds, logging fitness	9 Nov 2026
4	UCS, A* and both CSP variants solve all 30 small instances with node and backtrack counts	15 Nov 2026
5	Kill matrices for all 3 projects; one command produces the results table and all charts	19 Nov 2026

10. Expected Outcome and Risks

A working submission lets the user open the dashboard, pick a project and a time budget, and compare the chosen tests, coverage kept and time saved across methods. I expect simulated annealing to beat greedy by a few percentage points at tight budgets and to come within a few percent of the exact optimum, A* to expand far fewer nodes than UCS, and Kalman estimates to cut budget overruns compared with single-run timings. Risk 1: a project's tests fail locally or mutation analysis is too slow: I will switch to another candidate project and sample fewer mutants. Risk 2: exact search is too slow:I will reduce the sub-suite size.